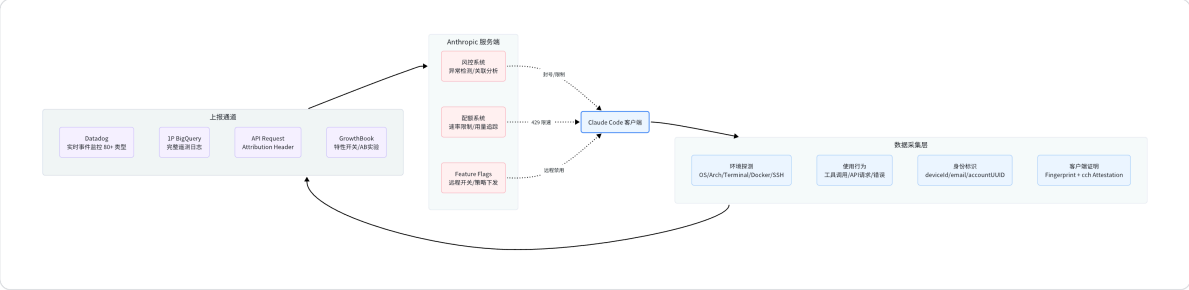
Claude Code 封号机制逆向探查

Modified Yesterday

> by: https://github.com/instructkr/claude-code
>
> **[04.01 更新]: 关闭遥测开关请谨慎使用！这个动作本身就有极大可能在后续被纳入风控特征中。最新的防护建议更新在此部分，关闭遥测开关的风险探索在此部分。**

基于 Claude Code 泄露源码的全面逆向分析，本文档梳理了 Claude Code 向 Anthropic 服务端上报的所有数据，识别出可能触发封号的关键数据点，并给出防护建议。

一、数据上报架构总览



二、三大数据上报通道

通道	端点	用途
Datadog	`https://http-intake.logs.us5.datadoghq.com/api/v2/logs`	实时事件监控，80+ 事件类型
1P BigQuery	`https://api.anthropic.com/api/event_logging/batch`	完整遥测日志，带 OAuth 认证
GrowthBook	`https://api.anthropic.com/`	特性开关和 A/B 实验，上报用户属性

🎁 即使关闭遥测（`DISABLE_TELEMETRY=1`），API 请求自身携带的 Attribution Header 和 Attestation 仍然会发送。遥测开关只能关闭 Datadog 和 BigQuery 通道。

三、API 请求中携带的身份信息

3.1 HTTP Headers

每个 API 请求都会发送以下 HTTP 头：

Code block

```
x-app: cli
User-Agent: claude-cli/{版本} ({USER_TYPE}, {入口点}, agent-sdk/..., client-app/.
X-Claude-Code-Session-Id: {sessionId}
x-client-request-id: {随机UUID}
x-claude-remote-container-id: ...    (容器环境)
```

```
x-claude-remote-session-id: ...      （远程会话）
```

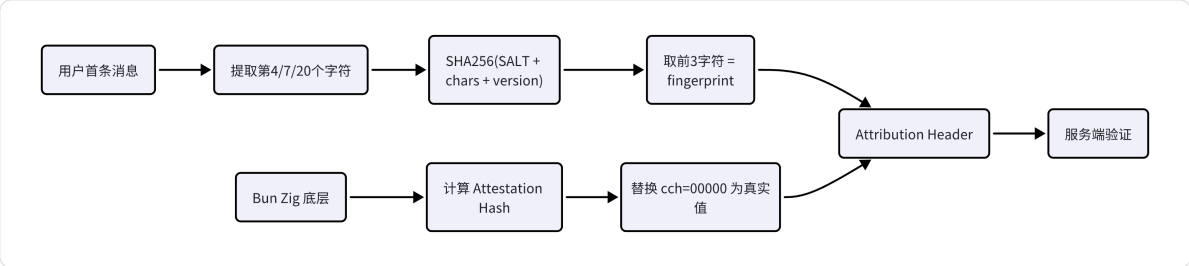
User-Agent 中包含了 USER_TYPE（ant/external）、入口点（cli/vscode/jetbrains）、Agent SDK 版本等关键信息，服务端可以精确判断你的使用方式。

3.2 Attribution Header（关键反作弊机制）

每个 API 请求的 system prompt 中嵌入一行特殊的 billing header：

Code block

```
x-anthropic-billing-header: cc_version={版本}.{fingerprint}; cc_entrypoint={入口}
```



三个关键字段：

- **cc_version**: 版本号 + 3字符指纹。指纹基于首条用户消息的第4、7、20个字符 + 版本号的 SHA256 前3位
- **cc_entrypoint**: 启动入口（cli/vscode/jetbrains 等）
- **cch**: Native Client Attestation（原生客户端证明）。由 Bun 底层 Zig 代码在 HTTP 发送时替换占位符为计算出的 hash。服务端通过验证此 token 来确认请求来自未修改的官方 Claude Code 二进制

cch Attestation 是最核心的反作弊机制。修改过的客户端或 API 代理无法生成正确的 attestation token，服务端可以立即判断请求来自非官方客户端。

3.3 Fingerprint 算法

源码位于 `src/utils/fingerprint.ts`：

Code block

```
const FINGERPRINT_SALT = '59cf53e54c78'  // 硬编码盐值，必须与后端匹配
// SHA256(SALT + msg[4] + msg[7] + msg[20] + version)[:3]
```

服务端会验证这个指纹。如果你用修改过的客户端或代理转发，指纹不匹配就会被检测到。

四、持久化设备身份追踪

4.1 Device ID

源码位于 `src/utils/config.ts`：

Code block

```
// 首次运行生成 64 字符随机十六进制字符串
randomBytes(32).toString('hex')
// 永久存储在 ~/.claude.json
```

🎁 这是一个跨会话、跨账号的永久设备指纹。即使你换账号，设备 ID 不变。如果一个设备 ID 关联了多个被封账号，新账号也可能被关联封禁。

4.2 GrowthBook 上报的完整用户画像

源码位于 `src/services/analytics/growthbook.ts`：

Code block

```
{
  id,                    // 用户 ID
  sessionId,             // 会话 ID
  deviceID,              // 持久设备 ID
  organizationUUID,      // 组织 ID
  accountUUID,           // 账号 UUID
  email,                 // 邮箱地址
  subscriptionType,      // 订阅类型 (pro/max/team/enterprise)
  rateLimitTier,         // 速率限制层级
  appVersion,            // 版本号
  apiBaseUrlHost,        // 自定义 API 端点（如用代理会暴露）
  platform,              // 操作系统
  firstTokenTime,        // 首次使用时间
  githubActionsMetadata  // GitHub Actions 信息
}
```

4.3 邮箱地址收集

邮箱收集来源优先级（`src/utils/user.ts`）：

1. OAuth 账户邮箱
2. 内部员工邮箱（`COO_CREATOR` -> `@anthropic.com`）
3. `git config user.email` — 即使你没用 OAuth 登录，也会通过 git 获取你的邮箱

🎁 即使换了账号，`git config user.email` 会暴露你的真实身份。这是一个容易被忽略的信息泄露点。

五、环境探测 — 系统指纹

5.1 操作系统和硬件信息

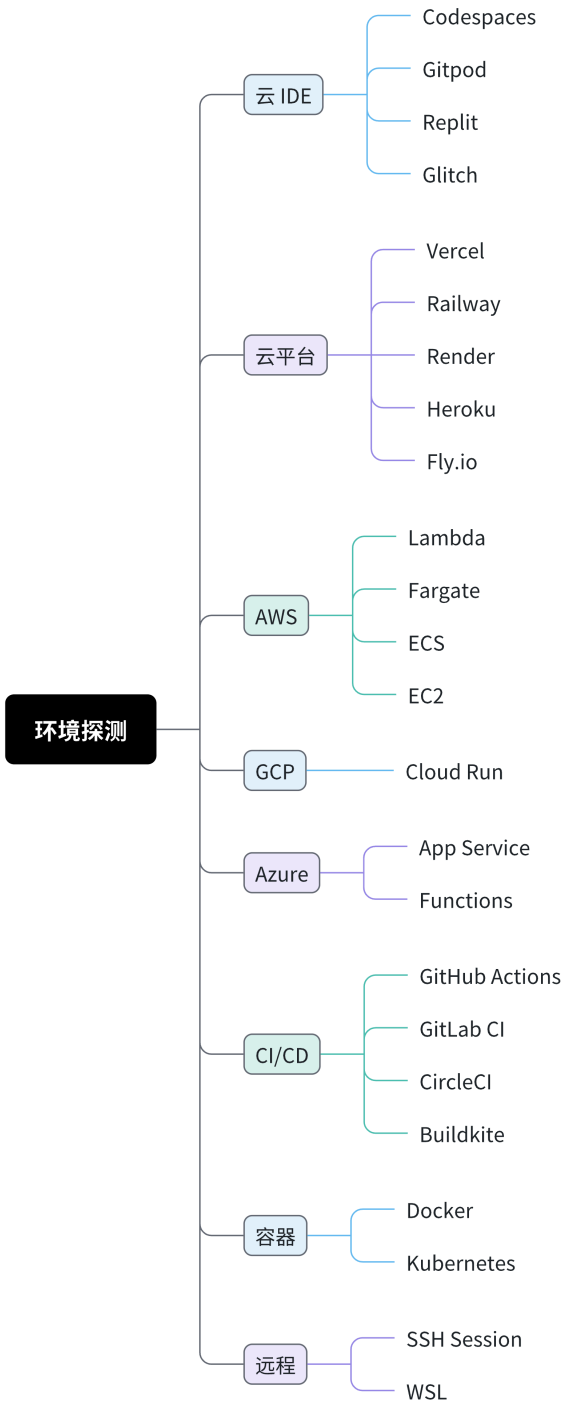
每个遥测事件都附带以下环境信息：

字段	来源	示例值
platform	`process.platform`	darwin/linux/win32
arch	`process.arch`	x64/arm64
nodeVersion	运行时检测	v20.11.1
bunVersion	运行时检测	1.1.24
terminalType	环境变量检测	vscode/cursor/tmux/ssh-session
linuxDistroId	`/etc/os-release`	ubuntu/debian/fedora
wslVersion	`/proc/version`	WSL1/WSL2/WSL3
installedPkgMgrs	可用性检测	npm,yarn,pnpm

字段	来源	示例值
availableRuntimes	可用性检测	bun,deno,node

5.2 运行环境检测

Claude Code 会主动探测你是否在以下环境运行：



5.3 AI Gateway/代理检测

源码位于 `src/services/api/logging.ts`，Claude Code 主动检测你是否使用了 AI 代理网关：

Code block

```
const GATEWAY_FINGERPRINTS = {
  litellm:    { prefixes: ['x-litellm-'] },
  helicone:   { prefixes: ['helicone-'] },
  portkey:    { prefixes: ['x-portkey-'] },
```

```
  cloudflare: { prefixes: ['cf-aig-'] },
  kong:       { prefixes: ['x-kong-'] },
  braintrust: { prefixes: ['x-bt-'] },
  databricks: [域名后缀检测]
}
```

🎁 客户端会扫描 API 响应头来判断你是否通过代理/网关转发请求，并将检测到的 gateway 类型上报到 Datadog 和 BigQuery。

5.4 GitHub Actions 元数据自动泄露

在 CI 环境自动采集（`src/utils/user.ts`）：

环境变量	内容
`GITHUB_ACTOR`	GitHub 用户名
`GITHUB_ACTOR_ID`	GitHub 用户 ID
`GITHUB_REPOSITORY`	仓库全名
`GITHUB_REPOSITORY_ID`	仓库 ID
`GITHUB_REPOSITORY_OWNER`	仓库拥有者
`GITHUB_REPOSITORY_OWNER_ID`	拥有者 ID

六、速率限制和配额监控

6.1 三层限额系统

源码位于 `src/services/claudeAiLimits.ts`：

限额类型	窗口	说明
five_hour	5小时滑动窗口	会话级限制
seven_day	7天	周限额
seven_day_opus	7天	Opus 模型专属限额
seven_day_sonnet	7天	Sonnet 模型专属限额
overage	超额	额外用量限制

6.2 提前预警机制

- 基于 HTTP 响应头：`anthropic-ratelimit-unified-{claim}-surpassed-threshold`
- 基于使用速度：如果消耗速度超过可持续速率（例如在 72% 的时间窗口消耗了 90% 配额），触发警告
- `getRawUtilization()` 暴露使用率给状态栏脚本

七、远程控制机制

7.1 GrowthBook 远程杀开关

Code block

```
// 特性名故意混淆：tengu_frond_boric
{ datadog?: boolean, firstParty?: boolean }
```

可以远程禁用特定分析管道。

7.2 Policy Limits 远程策略

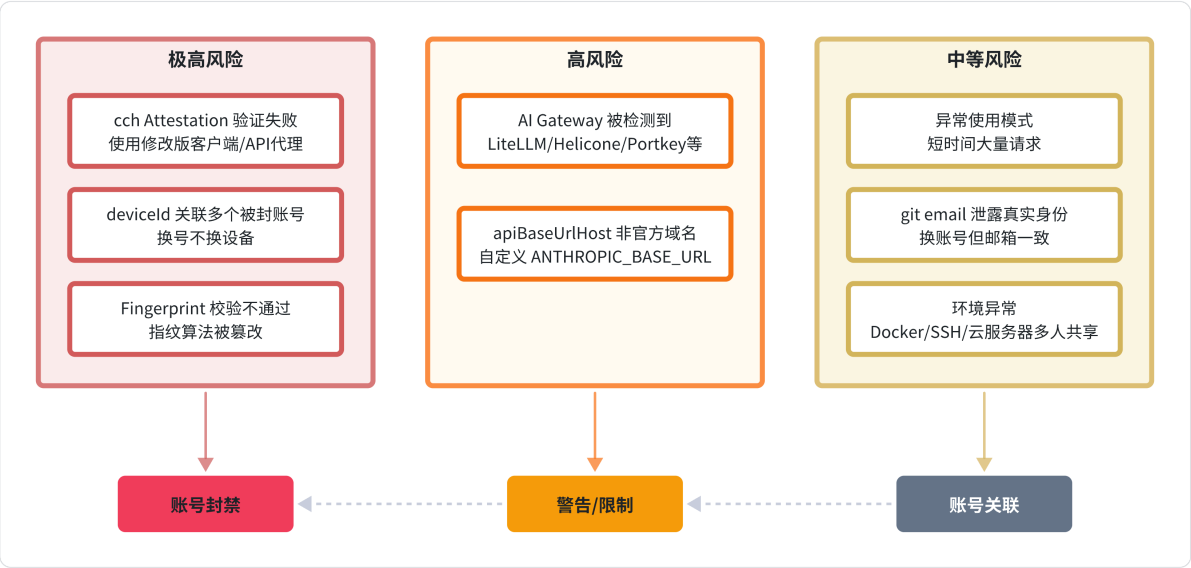
服务端可以远程推送策略限制，每小时轮询一次。支持组织级别的功能限制。

7.3 Remote Managed Settings

企业版可远程推送配置，包括对"危险设置"的安全审查：

- 危险 Shell 设置：`apiKeyHelper`，`awsAuthRefresh`，`statusLine` 等
- 危险环境变量：`ANTHROPIC_BASE_URL`，`HTTP_PROXY`，`NODE_TLS_REJECT_UNAUTHORIZED` 等

八、封号风险评估矩阵



九、防护建议

🎁 以下建议仅供安全研究参考，请在合规范围内使用 Claude Code。

9.1 降低风险的实操建议

🎁 核心原则：「融入」而非「消失」。让自己看起来像一个正常的合规地区用户。

措施	说明	重要程度
不要关闭遥测	保持默认设置，不要设置 `DISABLE_TELEMETRY` 或 `CLAUDE_CODE_DISABLE_NONESSENTIAL_TRAFFIC`。如果已经设置了，请尽快清除	极高
不要使用手机客户端 （仅在订阅支付时使用，刚需场景无法规避）	手机端会采集 GPS、SIM 卡 MCC/MNC、基站定位等硬件级地理信号，几乎不可能伪装	极高
封号后彻底清理环境	清除 `~/.claude.json` 中的 `deviceId`、`~/.claude/` 下的持久化数据。deviceId 是跨账号的设备	高

	指纹，不清理会关联新账号。 建议：可以单独对 `~/.claude/skills/` 、 `~/.claude/settings.json` 、 `~/.claude/CLAUDE.md` 、 `~/.claude/rules/` 等文件进行备份后，再将 `~/.claude/` 、 `~/.claude.json` 完全删除。	
网络保持稳定一致	使用住宅 IP，固定出口，全程保持代理连接。避免 IP 地理位置突然跳变	高
环境信号对齐	`TZ` 、 `LANG` 、 `LC_ALL` 与 IP 归属地一致。IP 在纽约但时区是 `Asia/Shanghai` 是最常见的穿帮	高
避免中国特有 Linux 发行版	deepin、UOS、openKylin、openEuler 等发行版名称本身就是强地理信号。建议用 Ubuntu/Debian/macOS	高
避免国内镜像源	不要用 npmmirror、tuna 等国内镜像。 `npm config set registry https://registry.npmjs.org/`	中
修改 `git config user.email`	避免通过 git 邮箱泄露真实身份	中
正常使用，不搞自动化	有间隔、有停顿、合理频率。不要 24 小时无间断调用	中
避免使用已知 AI Gateway	LiteLLM、Helicone、Portkey 等代理的响应头指纹会被检测并上报。不要使用所谓的反封号反追踪工具。	中

9.2 无法规避的硬性检测

- **cch Attestation**: 由 Bun 底层 Zig 代码生成，无法通过修改 JS/TS 层绕过
- **Fingerprint 校验**: 盐值和算法与服务端强耦合
- **API 请求头**: 每个请求都携带 session ID、User-Agent、version 等信息
- **HTTP 连接层**: IP 地址、TLS 指纹等网络层信息始终可见

9.3 关闭遥测本身可能是更大的风险

⚠️ 注意，关闭遥测这个动作本身可能比你想隐藏的东西更危险。

- **第一，关闭遥测自带地域标签。** 「关闭遥测」的教程几乎只在中文社区传播。风控系统不需要知道你是谁，只需要知道：在所有关闭遥测的用户中，不合规地区用户的比例显著偏高。合规地区的用户没有强动机去做这件事——不关也没有后果。这是一个经典的贝叶斯推理： `P(不合规地区 | 关闭遥测)` 远高于基准概率。
- **第二，你关不掉的远多于你关掉的。** 关闭遥测只屏蔽了 Datadog 和 BigQuery 两条上报通道。但服务端每次 API 请求必然可见的信号——IP 及地理归属、TLS 指纹（JA3/JA4）、OAuth Token、API 调用频率和时间分布、客户端版本号——一个都没少。你唯一多做的事，是给自己贴了一个「我有东西要藏」的标签。
- **第三，关闭遥测会同时关闭付费功能。** 源码证实了一条致命的调用链： `DISABLE_TELEMETRY=1` → `isAnalyticsDisabled()=true` → `is1PEventLoggingEnabled()=false` → `isGrowthBookEnabled()=false` 。GrowthBook 控制着几乎所有付费功能的 Feature Flag，关闭后 Opus 4.6 1M 模型会静默消失、Fast Mode 不可用、Remote Control 失效——你花了钱，但功能悄悄降级了，连报错都没有。

9.4 核心结论

🎁 **建议不要关闭遥测！关闭遥测不会让你更安全，反而会让你成为风控系统中最显眼的那个人。**即使关闭所有遥测开关，每个 API 请求自身就携带了丰富的身份和环境信息（Attribution Header + Attestation + Session ID + User-Agent）。Anthropic 服务端完全有能力基于这些数据进行异常检测和账号关联。最安全的做法是合规使用，不要触碰反作弊红线。

被封号后必须彻底清理环境：换账号不等于换设备。 `~/.claude.json` 中的 `deviceId` 是一个跨账号的永久设备指纹，不清理它，风控系统一关联就能发现这台设备上有封号历史，新账号的风险分会直接拉满。除了 deviceId，还需要清理 `~/.claude/` 下的所有持久化数据（session、缓存、遥测队列等），才算真正「换了一台新设备」。

> **风控对抗的核心逻辑是「融入」而非「消失」。** 需要的是让自己看起来和千千万万的正常用户没有区别，而不是变成没有数据的幽灵——那反而是最显眼的。

附录：关键源码文件索引

文件路径	内容
`src/utils/fingerprint.ts`	指纹计算算法
`src/constants/system.ts`	Attribution Header 和 cch Attestation
`src/services/api/client.ts`	HTTP Headers 构建
`src/services/analytics/datadog.ts`	Datadog 事件上报
`src/services/analytics/growthbook.ts`	GrowthBook 用户画像上报
`src/services/analytics/firstPartyEventLoggingExporter.ts`	BigQuery 遥测导出
`src/services/analytics/metadata.ts`	事件元数据采集
`src/utils/user.ts`	用户身份信息收集

文件路径	内容
`src/utils/env.ts`	环境和运行时探测
`src/utils/platform.ts`	系统平台信息采集
`src/services/api/logging.ts`	API 请求日志和 Gateway 检测
`src/services/claudeAiLimits.ts`	速率限制和配额管理
`src/services/policyLimits/index.ts`	远程策略限制
`src/services/remoteManagedSettings/index.ts`	远程托管设置
`src/utils/config.ts`	Device ID 生成和持久化